

Universidad de Buenos Aires

Facultad de Ingeniería

Informe de Trabajo Práctico Final

Abierto IA - Mercado Servicios

Materia: Introducción al Desarrollo de Software

Cátedra: Lanzillota

Año: 2025

Integrantes del Equipo:

Nombre y Apellido	Legajo
Francisco Ballerio	114777
Lautaro Soria	114698
Yessenia Valdivia	114385
Agustin Melfi	114508

Fecha de Entrega:

Fecha de entrega 5 de Diciembre de 2025

Índice

1. Resumen	2
2. Introducción	2
3. Solución Propuesta	2
3.1. Funcionalidades Clave	2
3.2. Decisiones de Diseño	3
4. Tecnología Utilizada	3
4.1. Backend	3
4.2. Frontend	3
4.3. Herramientas de Desarrollo	4
5. Dificultades Encontradas	4
6. Conclusión Final	4

1. Resumen

El presente informe detalla el desarrollo de **Mercado Servicios**, una plataforma web diseñada para conectar a proveedores de servicios profesionales (plomeros, electricistas, carpinteros, etc.) con clientes que requieren soluciones para el hogar. El sistema permite a los usuarios registrarse, buscar servicios mediante filtros avanzados, gestionar reservas en tiempo real y calificar la atención recibida. La solución implementa una arquitectura cliente-servidor desacoplada, utilizando Python (Flask) tanto para el backend como para el frontend, integrando una base de datos relacional MySQL y un sistema de autenticación seguro basado en JWT.

2. Introducción

En la actualidad, la contratación de servicios para el hogar suele caracterizarse por la informalidad, la falta de transparencia en los precios y la dificultad para verificar la reputación de los profesionales. Tanto los clientes como los proveedores carecen de una herramienta centralizada que gestione el ciclo de vida completo del servicio, desde el contacto inicial hasta la finalización del trabajo.

El objetivo de este Trabajo Práctico fue desarrollar una solución de software integral que resuelva esta problemática. **Mercado Servicios** nace como un "marketplace" de servicios que otorga visibilidad a los profesionales y seguridad a los clientes. El proyecto integra los conocimientos adquiridos en la materia, aplicando buenas prácticas de programación, control de versiones con Git/GitHub y metodologías ágiles para la gestión de tareas.

3. Solución Propuesta

La solución desarrollada es una aplicación web responsiva que gestiona tres roles principales: *Cliente*, *Proveedor* y *Administrador*.

3.1. Funcionalidades Clave

- **Autenticación y Autorización de Usuarios:** Registro e inicio de sesión seguros utilizando tokens JWT almacenados en cookies HttpOnly para prevenir ataques XSS. Diferenciación de permisos según el rol.
- **Catálogo de Servicios:** Los proveedores pueden crear, editar y eliminar sus servicios, especificando categorías, precios, zonas de cobertura (barrios) y disponibilidad horaria.
- **Búsqueda y Filtrado:** Los clientes pueden buscar servicios por palabras clave y filtrar por categoría, rango de precios y ubicación.
- **Sistema de Reservas:** Flujo completo de contratación que incluye la selección de fecha y hora, validación de disponibilidad para evitar superposiciones y gestión de estados (pendiente, confirmado, realizado, cancelado).

- **Confirmación mediante QR:** Implementación de un mecanismo de seguridad donde el sistema genera un código QR único para cada reserva. El proveedor debe escanear este código al finalizar el trabajo para marcar el servicio como Realizado”.
- **Sistema de Reseñas:** Una vez confirmado el servicio, el cliente puede calificar al proveedor con un puntaje (1-5 estrellas) y un comentario, alimentando la reputación del profesional.

3.2. Decisiones de Diseño

Se optó por una arquitectura donde el **Frontend** y el **Backend** están lógicamente separados y hosteados por separado. El Backend y la Base de Datos se encuentran alojados en la plataforma Pythonanywhere, mientras que el Frontend se encuentra en Render.

- El Backend expone una API RESTful que responde en formato JSON.
- El Frontend consume esta API utilizando la librería ‘requests’ de Python (patrón *Backend-for-Frontend*) y renderiza las vistas con Jinja2.
- Esta separación permite que, en el futuro, el backend pueda servir a una aplicación móvil sin necesidad de modificar la lógica de negocio.

4. Tecnología Utilizada

Para el desarrollo del proyecto se seleccionó un stack tecnológico robusto y moderno, alineado con los requerimientos de la cátedra:

4.1. Backend

- **Lenguaje:** Python 3.
- **Framework Web:** Flask (microframework).
- **Base de Datos:** MySQL 8.0.
- **Driver de BD:** mysql-connector-python.
- **Autenticación:** Flask-JWT-Extended para manejo de tokens y sesiones.

4.2. Frontend

- **Motor de Plantillas:** Jinja2 (integrado en Flask) para renderizado dinámico de HTML.
- **Estilos:** CSS3 nativo, con un diseño modular inspirado en Mercado Libre, utilizando variables CSS y Flexbox/Grid.
- **Interactividad:** JavaScript (Vanilla JS) para validaciones de formularios, manejo de modales y lógica del lado del cliente.

- **Librerías Adicionales:** `qrcode` (Python) para la generación dinámica de códigos de confirmación.

4.3. Herramientas de Desarrollo

- **Control de Versiones:** Git y GitHub.
- **Testing de API:** Bruno (se crearon colecciones de prueba para validar los endpoints de Auth, Servicios y Reservas).
- **Entorno:** Scripts de automatización en Bash (`start.sh`, `stop.sh`) para la gestión de procesos.

5. Dificultades Encontradas

Durante el desarrollo, el equipo enfrentó y superó los siguientes desafíos:

1. **Manejo de Fechas y Horarios:** La validación de disponibilidad para evitar reservas duplicadas requirió una lógica compleja para comparar rangos horarios, duraciones de servicio y fechas ya reservadas.
2. **Serialización de Datos:** La base de datos MySQL retorna tipos decimales (para precios y promedios) que no son serializables automáticamente a JSON, lo que requirió conversores manuales en las respuestas de la API.
3. **Conflictos de Merge:** Al trabajar varios integrantes sobre los mismos archivos (especialmente en rutas y templates base), surgieron conflictos en Git que se mitigaron mejorando la comunicación y dividiendo el trabajo por módulos funcionales.

6. Conclusión Final

El desarrollo de **Mercado Servicios** permitió al equipo integrar de manera práctica los conceptos teóricos de la ingeniería de software. Se logró construir un producto funcional que cumple con los requisitos obligatorios: persistencia en base de datos, arquitectura cliente-servidor y uso de control de versiones.

La implementación de funcionalidades avanzadas, como la generación de códigos QR y el sistema de reputación, aportó un valor agregado significativo al producto final. Asimismo, la experiencia destacó la importancia de una buena definición de la API antes de comenzar la implementación del frontend, y cómo las metodologías ágiles facilitan la adaptación ante cambios y dificultades técnicas. El resultado es una plataforma escalable y robusta, lista para ser desplegada en un entorno productivo.